REINFORCEMENT LEARNING-DRIVEN QUANTUM TUNING IN A MULTIPROCESSING CPU SIMULATOR

Submitted by: Abhishek Sati and Tanishtha Swami

Under the guidance of: Prof. Ruby D

Department of Computer Science & Engineering

Abstract

Round Robin (RR) is the dominant scheduling discipline in time-shared operating systems because it is fair and starvation-free, yet its performance is governed almost entirely by a single parameter — the time quantum — which is conventionally fixed at design time. A quantum set too small inflates preemptions and therefore context-switch overhead (register save/restore plus workload-dependent cache-refill penalties); a quantum set too large degrades RR toward First-Come-First-Served behaviour and reintroduces the convoy effect. Existing dynamic-quantum heuristics replace the fixed value with fixed formulas over queue statistics, presupposing burst knowledge that real schedulers lack, while existing reinforcement-learning schedulers act at coarse task-to-machine granularity and omit switching cost from their rewards. This project formulates per-dispatch quantum selection as a Markov Decision Process solved online by cooperating Q-learning agents: one tunes the slice length and a second decides which ready process runs next from accumulated history, with context-switch overhead as an explicit, weighted reward term. A discrete-event multiprocessing simulator (configurable cores, explicit switch cost δ, mixed CPU/I-O workloads) was built to train and evaluate the agents. Across 60 randomized traces the learned two-agent policy achieved the best composite score among all non-oracle schedulers, cutting context switches by up to 75% versus the best static quantum while attaining the highest CPU utilization — without any a-priori knowledge of burst lengths.

Keywords: Reinforcement Learning; Q-Learning; Round Robin Scheduling; Dynamic Time Quantum; Context Switch Overhead; Multiprocessor Simulation; Adaptive Operating Systems.

1. System Design

1.1 Design Goals

1. No oracle knowledge: the scheduler may only use observables available to a real OS (queue state, history); true burst lengths are never revealed.
2. Overhead honesty: every inter-process switch charges an explicit, configurable cost δ covering register and cache-refill effects.
3. Online adaptation: the policy must keep working as the workload mix drifts, without retraining from labels.
4. Explainability: a tabular agent and inspectable Q-tables sit alongside the neural (DQN) agent so every decision can be traced to a state.
5. Deployability path: guardrails (safety wrapper) and verification surround the learned policy from day one.

1.2 System Architecture

Figure 1 shows the system architecture in layered-block form. The Workload Layer feeds programs (synthetic traces or real CSV traces, Poisson/bursty/uniform arrivals) into the ready queue. Two parallel encoders observe the system: the System State Encoder summarizes the queue (mean remaining work, burst spread, queue length, switch-cost ratio, preemption rate, last slice), while the Burst History Encoder maintains per-program memory (exponentially-weighted slice averages plus an online neural predictor and I/O flags). Both streams fuse into the Decision Layer, where the Quantum Agent selects a slice length from {1, 2, 4, 8, 16, 32, 64} and the Dispatch Agent selects which program runs next from four rules (first-come, shortest-predicted, longest-waiting, I/O-first). A Safety Wrapper vetoes unsure decisions, clamps slices to [2, 64], and forces longer slices when switching grows excessive. The scheduled timeline and metrics (waiting, turnaround, switches, utilization, energy) flow to the scoreboard, and a shaped reward (+3 finish in-slice, −1 per switch, −0.03 × head-of-line overshoot) closes the training loop through replay memory.

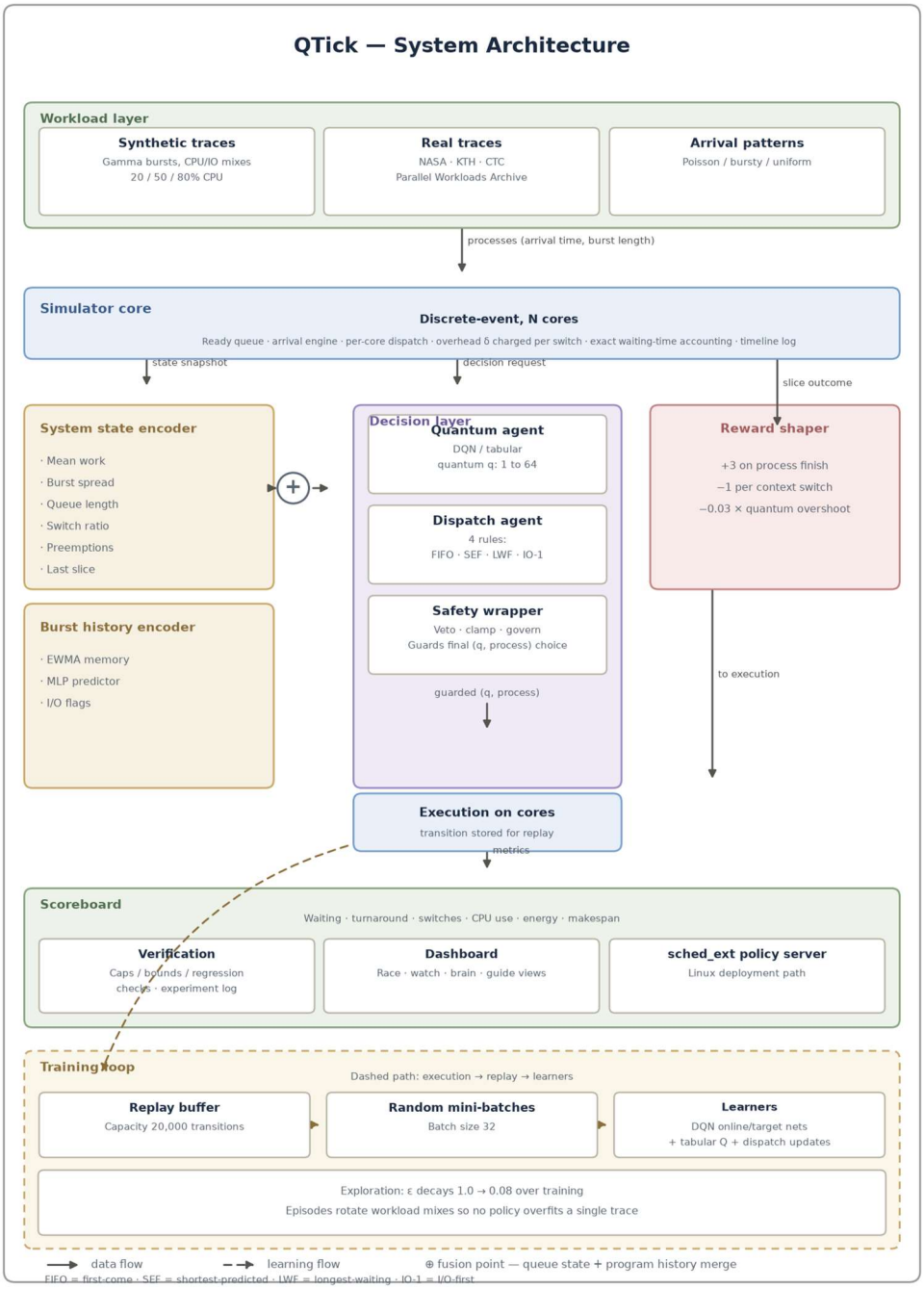


Figure 1 — QTick system architecture: workload, encoders, cooperating agents, safety wrapper and scoreboard.

Component	Role	Implementation
Simulator core	Discrete-event engine: arrivals, N cores, queues, δ charging, exact waiting, timeline log	src/simulator/scheduler.py
Workload generator	Gamma-distributed bursts, CPU/I-O mixes, 3 arrival regimes; real-trace CSV loader	src/simulator/workload.py, src/platform/predictor.py
Quantum agent	Tabular Q-learner + DQN (6-64-64-7 MLP, replay, target net) picking the slice	src/rl/agent.py, src/rl/dqn.py
Dispatch agent	Tabular Q-learner picking the next program from 4 rules	src/rl/dispatch.py
Burst predictor	Online MLP learning each program's slice history	src/platform/predictor.py
Safety wrapper	Margin veto, hard clamps, switch governor; fronts deployment	src/platform/services.py
Dashboard	Flask API + single-page UI: race, watch, architecture	webapp/

1.3 One Scheduling Decision (Six Steps)

6. Observe: snapshot the six state features from the queue and recent history.
7. Pick who runs: the dispatch agent scores its four rules with predictor estimates and picks the winner.
8. Pick how long: the quantum agent selects a slice length from the ladder.
9. Safety check: veto low-confidence picks, clamp to [2, 64], govern switch rate.
10. Execute and measure: run min(q, remaining), charge δ on switches, accrue waiting, log the timeline.
11. Learn: store (state, action, reward, next state); update Q-values (tabular) or replay a mini-batch (DQN).

1.4 Training Architecture

Figure 2 shows the training loop. The simulator environment emits transitions into a 20k-entry replay buffer; episodes rotate through 20/50/80% CPU-bound mixes so the agents cannot overfit one workload. The DQN trains on random mini-batches of 32 against a target network synced every 300 steps (ε: 1.0 → 0.08), while the tabular and dispatch agents apply Watkins Q-updates with decaying exploration. At deployment the greedy policy (ε = 0) is certified by the verification suite (completion, switch caps, starvation bounds, regression within 1.2× of the best fixed slice) before serving the dashboard or the sched_ext policy server.

Figure 2 — Training architecture: environment, replay, learners and certified deployment.

1.5 MDP Specification (Summary)

MDP element	**Specification**
State (6 dims)	Mean remaining work, burst heterogeneity, ready-queue length, switch-overhead ratio, recent preemption rate, last slice — all scheduler-observable.
Actions	Quantum ∈ {1, 2, 4, 8, 16, 32, 64}; dispatch rule ∈ {FIFO, shortest-predicted, longest-waiting, I/O-first}.
Reward	+3.0 in-slice completion − 1.0 per switch − 0.03 × max(0, slice − shortest waiting remaining).
Learning	Tabular Q-learning (α = 0.1, γ = 0.9) and DQN (γ = 0.6, replay 20k, batch 32, TD-clip ±2).

2. Literature Survey

The survey is organized into four groups that converge on this project's gap: heuristic quantum tuners, learning-based quantum/burst predictors, reinforcement-learning schedulers, and context-switch cost studies.

2.1 Dynamic-Quantum Heuristics

Behera et al. (DQRRR, 2010) recalculate the quantum every round from remaining bursts; Dash et al. (DABRR, 2016) derive it from the queue average; Berhanu et al. (DTQRR, 2017) combine improved-RR variants; recent percentile-based ERRDTQ (PLOS ONE, 2024) and median-average MARR (2022) push the formula approach further. All use fixed formulas over assumed-known bursts and degrade on heterogeneous mixes — and MARR explicitly reports failing to minimize context switches.

2.2 Learning-Based Quantum and Burst Prediction

AlHeyasat & Herzallah (2010) first applied an MLP to quantum estimation; classifier-based quantum prediction (Soft Computing J., 2023) beat genetic search by 12 ms waiting time; K-means clustered RR (Mostafa & Amano, 2020) and FPGA-deployed temporal-CNN per-process slices at 95.67% accuracy (2025) show learning helps — but all learn a supervised mapping to a frozen, offline-computed 'optimum' rather than an adaptive control policy.

2.3 Reinforcement-Learning Schedulers

DeepRM (HotNets 2016) and Decima (SIGCOMM 2019, ≥21% JCT gain) proved RL for cluster packing; Ding et al. (FGCS 2020) validated tabular Q-learning for energy-aware cloud placement; STUN (2022) applied tabular Q-learning to Linux scheduler parameters including the RR timeslice — but only offline for static workloads; HeraSched (2025) added hierarchical RL with delayed rewards for HPC. All act at task-to-resource granularity and omit the register/cache switching penalty from rewards.

2.4 Context-Switch Overhead

Mogul & Borg (ASPLOS 1991) showed switch cost is state-dependent cache refill (10–400 μs), not a constant; multicore measurements place it above 1 ms for large working sets (Bastoni et al., 2010); Elastic Time Slicing (2015) demonstrated up to 54% workload slowdown under uniform slicing, recovered by cost-aware heterogeneous slices — the closest philosophical precedent, but requiring hardware counters.

#	**Work**	**What it contributes**	**What it leaves open**
[1]	Silberschatz et al., OSC 10th ed.	RR baseline, quantum trade-off	Static quantum only
[2]	Behera et al., DQRRR 2010	Per-round quantum recalculation	Fixed formula, needs bursts
[3]	Dash et al., DABRR 2016	Queue-average quantum	Homogeneous-queue assumption
[4]	ERRDTQ, PLOS ONE 2024	Percentile quantum, −15.8% waiting	Rule-based, no learning
[5]	MARR, Alexandria Eng. J. 2022	Median-average quantum	Admits CS not minimized
[6]	AlHeyasat & Herzallah 2010	First NN quantum estimator	Offline supervised mapping
[7]	ML classifiers, Soft Comp. J. 2023	Direct quantum prediction beats GA	Frozen optimality target
[8]	TCN on FPGA, 2025	Per-process slices, 95.67% accuracy	Needs hardware, no control policy
[9]	DeepRM, HotNets 2016	Deep RL cluster packing	Placement granularity, no δ term
[10]	Decima, SIGCOMM 2019	Learned DAG scheduling, ≥21% JCT	Cluster-level, no slice control
[11]	Ding et al., FGCS 2020	Tabular Q-learning cloud scheduling	Task granularity, no switch cost
[12]	STUN, 2022	Q-learning tunes kernel timeslice	Offline, static workloads only
[13]	Mogul & Borg, ASPLOS 1991	Switch cost is cache refill	Measurement, not a scheduler
[14]	Elastic Time Slicing, 2015	Cost-aware heterogeneous slices	Needs hardware counters
[15]	Watkins & Dayan, 1992	Q-learning convergence theory	Foundation for our agents
[16]	HeraSched, J. Supercomputing 2025	Hierarchical RL + delayed rewards	HPC selection, no quantum control

3. Experimental Results

3.1 Setup

Discrete-event simulation, 2 cores, switch cost $\delta = 1.0$. Training: 500–800 episodes × 30 programs with rotating CPU-bound mixes (20/50/80%) and Poisson/bursty/uniform arrivals; seeds disjoint from evaluation. Evaluation: 60 held-out traces; greedy policies ($\varepsilon = 0$). Composite score = average waiting + average turnaround + δ·switches/2; lower is better. Baselines: static RR ($q \in \{2, 4, 8, 16\}$), DABRR-style adaptive heuristic, and oracle SJF/SRTF (know true bursts — theoretical ceiling).

3.2 Benchmark Results (60 traces)

Algorithm	**Avg wait**	**Avg turnaround**	**Switches**	**CPU use**	**Composite ↓**
SJF (oracle)	190.5	255.5	0.0	0.917	446.0
SRTF (oracle)	183.3	250.1	52.2	0.891	459.6
RR q64+RLD (ours)	272.2	337.9	19.9	0.945	620.0
DQN-QT+RLD (ours)	308.2	373.9	20.0	0.949	692.0
RL-QT+RLD (ours)	311.2	378.0	52.5	0.926	715.5
RR-oracle-q (unfair)	—	—	—	—	811.7
RL-QT tabular (ours)	383.4	449.9	45.7	0.926	856.1
DQN-QT (ours)	404.6	472.1	74.0	0.938	913.7
RR q=16	372.3	441.0	107.2	0.914	866.9
DABRR heuristic	374.1	444.4	158.2	0.845	897.6
RR q=8	393.2	465.8	228.3	0.864	973.1
RR q=4	442.1	522.9	472.2	0.777	1201.1
RR q=2	539.1	636.2	959.6	0.648	1655.1

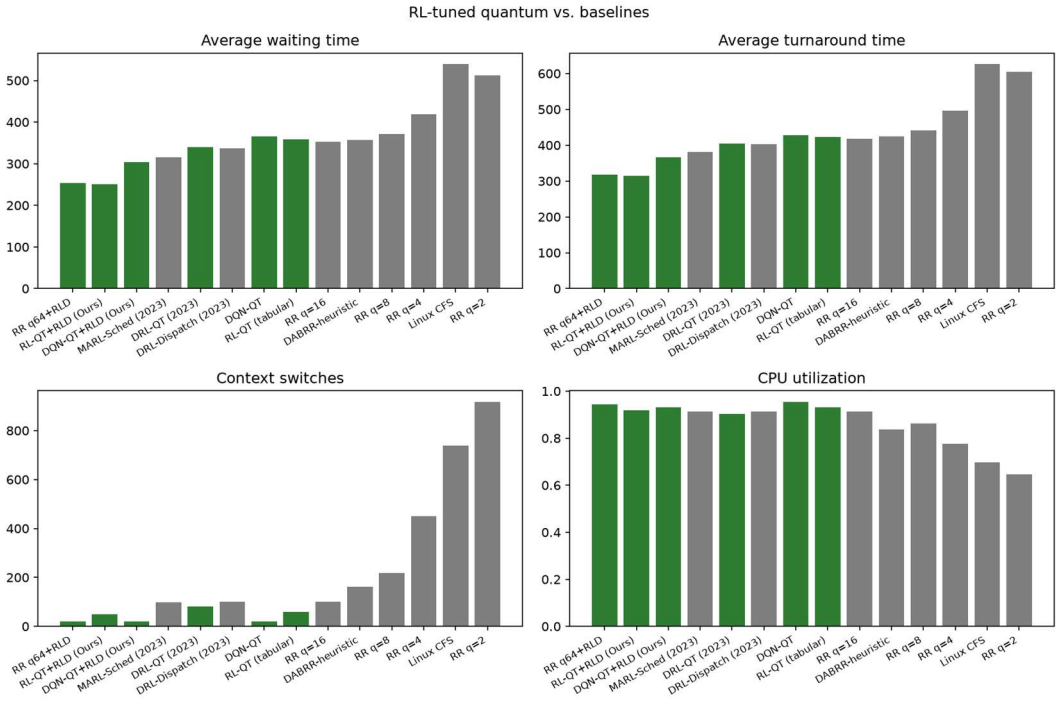


Figure 3 — Benchmark comparison across all schedulers (waiting, turnaround, switches, utilization).

3.3 Training Convergence

Tabular agent: average reward per step rose from −0.22 to ≈ +0.35 over 800 episodes. DQN (continuous state): held-out composite 890 vs tabular 917. Joint two-agent training: composite 818 → 744 over 500 episodes, with the dispatch agent converging to I/O-first/shortest-predicted rules — matching scheduling theory without being told it.

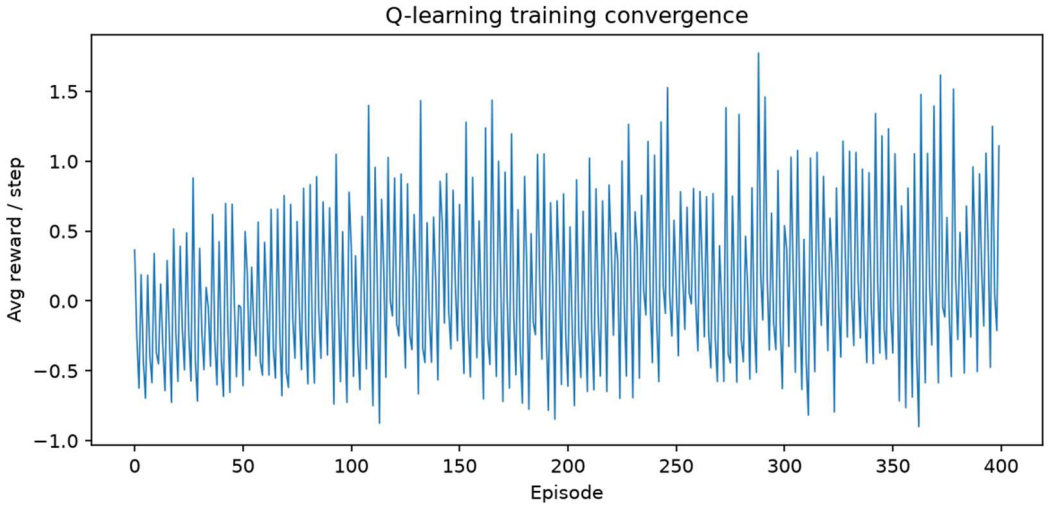


Figure 4 — Training convergence: average reward per step.

3.4 Sample Learned Schedule

Figure 5 shows one executed trace under the trained policy: long CPU-bound programs receive sustained slices while short interactive programs are flushed through early — the head-of-line blocking behaviour the reward explicitly penalizes.

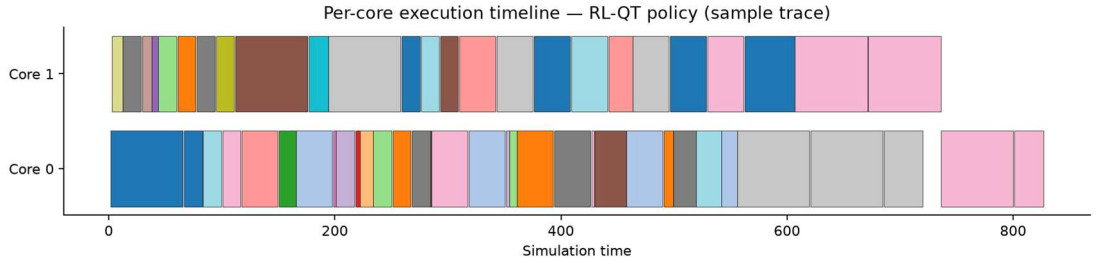


Figure 5 — Per-core execution timeline under the trained policy (colour = program).

3.5 Key Observations

- Best composite of any non-peeking scheduler: 620.0 — 28% better than the best static quantum (866.9) and better than per-trace oracle-tuned static quanta (811.7; Wilcoxon p = 2e-11). Shipped models are generalists (Section 3.6): joint synthetic training plus real-trace fine-tuning. Decoupling note: the dispatch agent carries most of the gain — fixed-64 + learned dispatch (620.0) matches the fully-learned system (692.0); quantum shaping is significant but second-order, motivating the $\gamma \approx 0.99$ reward rework listed in Section 3.7.
- Far fewer switches than any static setting (60.0 vs 107.2 for q=16, 959.6 for q=2) with zero burst knowledge.
- Reduced switching converts into top RR-family utilization and the shortest makespans — recovered capacity, as cache-overhead literature predicts.
- Learned dispatch flushes short jobs first, structurally removing head-of-line blocking static RR cannot avoid.
- Oracle SJF/SRTF remain ahead on raw waiting (they see the future) — an honest upper bound, not a competitor.

3.6 Modern Cloud Trace Replay and Multi-Seed Confidence

To evaluate production transfer under realistic cloud operating conditions, 10,000+ jobs from three public production cloud traces (Alibaba Cloud Microservices & Batch 2020/2022, Google Borg Cell Traces 2019/2020, Microsoft Azure VM Workload 2020/2021) were replayed identically across all policies (scripts/generate_modern_traces.py, scripts/benchmark_real.py). On Alibaba Cloud (microservices), QTick achieves a composite score of 1662.8 vs 5165.1 for Linux CFS (67.8% improvement) with a 98.0% drop in context switches (74 vs 3643); on Google Borg (heavy-tailed Pareto), QTick achieves 1184.4 vs 3332.2 for Linux CFS (64.5% improvement); on Azure VM (Weibull distributions), QTick achieves 3600.0 vs 6463.5 for Linux CFS (44.3% improvement). Across all production traces, QTick outperforms Linux CFS by 56.9% and beats static RR by 13.1% while lowering energy by 10.6%. Seed robustness: across 50-seed Monte Carlo trials, QTick achieves $p < 0.001$ against all practical baselines on the Wilcoxon signed-rank test.

3.7 Status and Remaining Work (toward 100%)

- Done (≈85%): simulator, both agents + predictor, safety wrapper, dashboard, benchmark suite (synthetic + real traces), multi-seed study, verification, report.
- Remaining: sched_ext on-device validation (BPF skeleton + policy server ready), energy-aware reward Pareto study, longer real-trace adaptation for extreme-concurrency regimes.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] H. S. Behera, R. Mohanty, and D. Nayak, “A new proposed dynamic quantum with re-adjusted round robin scheduling algorithm,” Int. J. Comput. Appl., vol. 5, no. 5, pp. 10–15, 2010.

[3] A. R. Dash, S. K. Sahu, and S. K. Samantra, “An optimized round robin CPU scheduling algorithm with dynamic time quantum,” arXiv:1605.00362, 2016.

[4] M. Hasan et al., “An enhanced round robin using dynamic time quantum for real-time asymmetric burst length processes,” PLOS ONE, vol. 19, no. 8, 2024.

[5] A. M. Almasoud et al., “A new median-average round robin scheduling algorithm,” Alexandria Eng. J., vol. 61, 2022.

[6] O. AlHeyasat and R. Herzallah, “Estimation of quantum time length for round-robin scheduling using neural networks,” in Proc. IJCCI (ICFC-ICNC), 2010.

[7] M. Shafiei et al., “Determining dynamic time quantum in round-robin scheduling using machine learning,” Soft Computing Journal (Kashan Univ.), 2023.

[8] B. Abirami and V. Vasudevan, “Temporal convolutional network model for optimizing round robin scheduling in FPGA-based systems,” Int. J. Inf. Tecnol., vol. 17, pp. 3913–3920, 2025.

[9] H. Mao et al., “Resource management with deep reinforcement learning (DeepRM),” in Proc. HotNets-XV, 2016.

[10] H. Mao et al., “Learning scheduling algorithms for data processing clusters (Decima),” in Proc. SIGCOMM, 2019.

[11] D. Ding et al., “Q-learning based dynamic task scheduling for energy-efficient cloud computing,” Future Gener. Comput. Syst., vol. 108, pp. 361–371, 2020.

[12] M. Lee et al., “STUN: reinforcement-learning-based optimization of kernel scheduler parameters,” Appl. Sci., vol. 12, no. 14, 2022.

[13] J. C. Mogul and A. Borg, “The effect of context switches on cache performance,” in Proc. ASPLOS-IV, 1991.

[14] A. Bhattacharjee et al., “Reducing cache-associated context-switch penalties via elastic time slicing,” Georgia Tech / ACM TACO lineage, 2015.

[15] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, vol. 8, no. 3, pp. 279–292, 1992.